



DIPARTIMENTO DI INFORMATICA

Corso di laurea Basi di Dati I

Anno Accademico 2023/2024

Progettazione e Sviluppo di un DATABASE

Docente:

Silvio Barra

Autore:

Alessandro Virgilio N86004878

Sommario

1	PROGETTAZIONE CONCETTUALE.....	4
1.1	Analisi dei requisiti	4
1.2	SCHEMA CONCETTUALE	5
1.3	DIZIONARIO DELLE ENTITA'	5
1.4	DIZIONARIO DELLE ASSOCIAZIONI	7
2	Ristrutturazione	8
2.1	Analisi ridondanze	8
2.2	Analisi delle generalizzazioni.....	8
2.3	Chiavi primarie	8
2.4	Diagramma UML.....	9
		9
2.5	Diagramma E-R	10
2.6	Dizionario classi	10
2.7	Dizionario associazioni	11
3	Modello logico	12
4	Progettazione fisica	13
4.1	Dizionario dei vincoli.....	13
4.2	Creazione database	13
4.3	Creazione tabelle	13
4.3.1	Tabella utente	13
4.3.2	Tabella pagine	14
4.3.3	Tabella modifiche	14
4.3.4	Tabella versioni	15
4.3.5	Tabella frasi.....	16
4.3.6	Tabella collegamenti	16
4.3.7	Tabella notifiche	17
4.4	Trigger ed event	17
4.4.1	Trigger per proposta modifica	17
4.4.2	Trigger accettazione e rifiuto modifica	18
4.4.3	Trigger controllo dati utente	19
4.4.4	Trigger controllo ruolo per modifica	20
4.4.5	Trigger collegamento e controllo frasi inserimento	21
4.4.6	Trigger collegamento e controllo frasi aggiornamento	23
4.4.7	Trigger per controllo cronologia modifiche	25
4.4.8	Trigger per controllo proprietario pagina	26

4.4.9	Trigger per gestione modifica da proprietario	26
4.4.10	Trigger per creazione versione pagina N1	27
4.4.11	Event creazione collegamenti.....	28
4.4.12	Event aggiornamento storico collegamenti	29
4.4.13	Event controllo pagina cancellata.....	30
4.4.14	Event controllo ruolo utente sbagliato	31
5	APPLICATIVO.....	32
5.1	Login.....	32
5.2	Registrazione	33
5.3	Menu	34
5.4	Utente.....	35
5.5	Pagine.....	36
5.6	Notifiche	37
5.7	Modifiche.....	38
5.8	Versioni.....	39

1 PROGETTAZIONE CONCETTUALE

1.1 Analisi dei requisiti

Come classe principale avremo la classe UTENTE che terrà traccia di tutti i dati degli utenti registrati al sistema di wiki e questa classe sarà quella che permette il totale funzionamento della wiki come la creazione e la modifica delle pagine di quest'ultima.

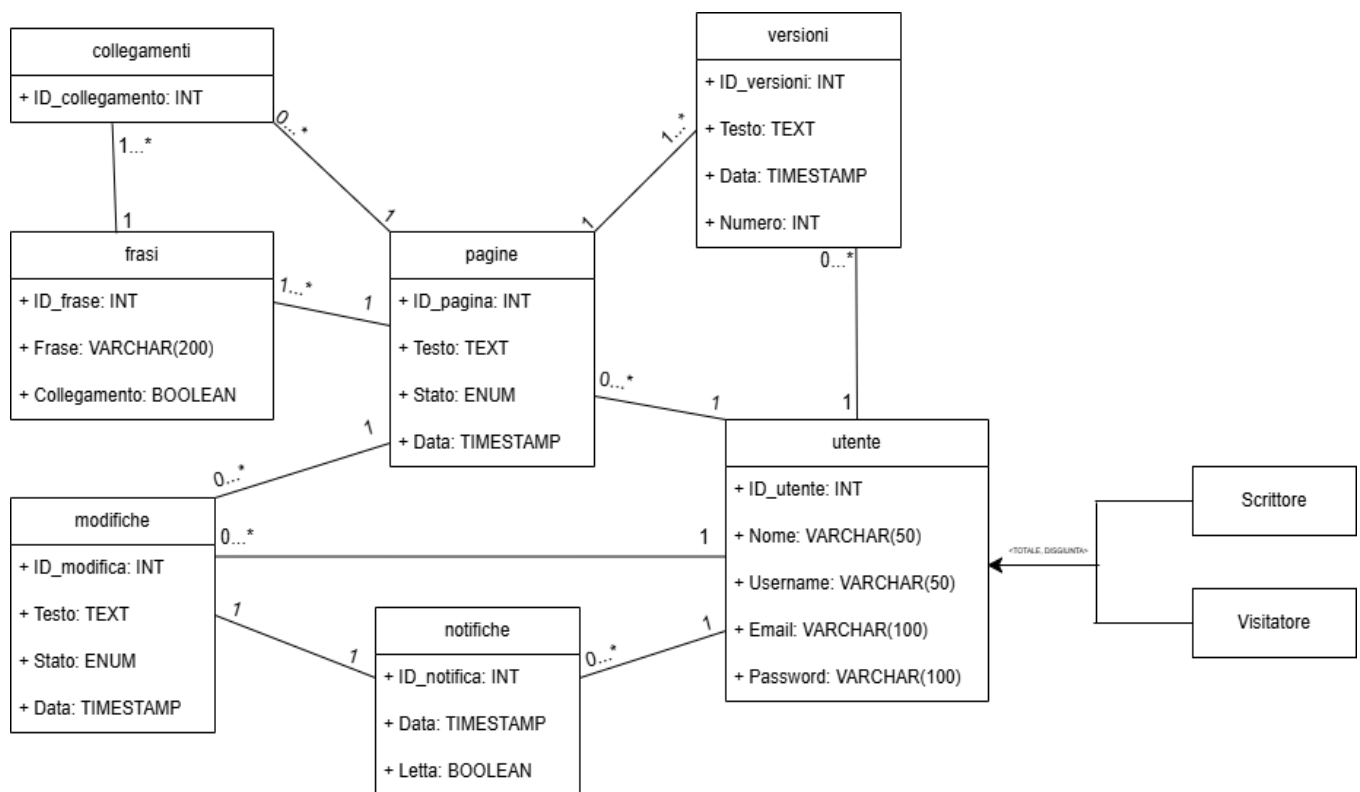
I dati presenti all'interno della classe utente saranno:

- ID_utente ovvero un ID univoco che viene generato automaticamente non appena l'utente si registra;
- Nome il nome dell'utente stesso che non può essere NULLO;
- Username dell'utente stesso anche lui unico che non può essere NULLO;
- Email ovvero la mail dell'utente che sarà necessaria solo se l'utente ha come ruolo scrittore;
- Password ovvero la password dell'utente che sarà obbligatoria solo nel caso in cui l'utente ha come ruolo scrittore;
- Ruolo il ruolo che ricopre l'utente che può essere o Visitatore ovvero un utente senza privilegi per la creazione e la modifica delle pagine, Scrittore un utente che ha i privilegi per la creazione e la modifica delle proprie pagine che ovviamente potranno essere più di una e potrà anche proporre la modifica di alcune pagine create dagli altri utenti che decideranno se approvarle oppure no. Di default il ruolo sarà visitatore;

La nostra Wiki conterrà le pagine e tutte le sue versioni precedenti, non appena verrà creata una pagina in versioni comparirà la pagina come prima versione e ogni volta che viene modificata verrà inserita la pagina modificata anche in versioni non sovrascrivendo la vecchia versione in modo da avere uno storico di tutte le versioni. Ogni qualvolta un'utente propone una modifica, al creatore della pagina arriverà una notifica con la pagina riferita alla modifica e sarà quest'ultimo a decidere se accettarla e quindi sovrascrivere la pagina o rifiutarla, dovranno essere accettate o rifiutate prima le modifiche più vecchie, tutto questo verrà gestito da trigger interni al **database**.

Ogni pagina avrà un proprio contenuto con all'interno delle parole o frasi chiave contrassegnate con [] che collegheranno la pagina ad un'altra nella wiki che approfondisce la parola o frase di riferimento.

1.2 SCHEMA CONCETTUALE



1.3 DIZIONARIO DELLE ENTITA'

ENTITA'	DESCRIZIONE	ATTRIBUTI
UTENTE	Contiene le informazioni generali di tutti gli utenti iscritti nella wiki	<i>ID_utente</i> (INT) Identificativo univoco dell'utente; - Nome(VARCHAR (50)) Nome dell'utente; - Username(VARCHAR (50)) Username dell'utente; - Email(VARCHAR (50)) Email dell'utente; - Password(VARCHAR(100)) Password dell'utente; - Ruolo(ENUM) Il ruolo dell'utente.
PAGINE	Contiene le informazioni riguardanti tutte le pagine presenti nella wiki	<i>ID_pagina</i>(INT) Identificativo univoco della pagina; - Titolo(VARCHAR(100)) Il titolo della pagina; - Testo(TEXT) Il testo presente nella pagina;

		• <i>Data(TIMESTAMP)</i> la data di creazione della pagina.
MODIFICHE	Contiene le informazioni riguardante le modifiche proposte, accettate e rifiutate	• <i>ID_modifica(INT)</i> Identificativo univoco della modifica; • <i>Testo(TEXT)</i> Il testo proposto per la modifica; • <i>Stato(ENUM)</i> Indica lo stato nella quale si trova la modifica; • <i>Data(TIMESTAMP)</i> La data di creazione della modifica.
VERSIONI	Contiene tutte le versioni delle pagine sovrascritte della wiki	• <i>ID_versioni(INT)</i> Identificativo univoco della versione della pagina; • <i>Testo(TEXT)</i> Il testo della versione della pagina; • <i>Data(TIMESTAMP)</i> La data di creazione della versione; • <i>Numero(INT)</i> Il numero della versione.
FRASI	Contiene tutte le frasi di una pagina divise per frasi collegate e non	• <i>ID_frase(INT)</i> Identificativo univoco della frase; • <i>Frase(VARCHAR(200))</i> La frase presa dal testo della pagina; • <i>Collegamento(BOOLEAN)</i> Indica se la frase è collegata o no;
COLLEGAMENTI	Contiene la scheda di tutte le frasi e le pagine collegate	• <i>ID_Collegamento(INT)</i> Identificativo univoco del collegamento;
NOTIFICHE	Contiene tutte le notifiche riguardanti le modifiche proposte	• <i>ID_notifica(INT)</i> Identificativo univoco della notifica; • <i>Data(TIMESTAMP)</i> La data di arrivo della notifica; • <i>Letta(BOOLEAN)</i> Indica se la notifica è stata letta o no.

1.4 DIZIONARIO DELLE ASSOCIAZIONI

ASSOCIAZIONI	DESCRIZIONE
CREA	Associazione 1 a N tra utente e pagine perché un utente può creare più di una pagina, ma una pagina non può essere creata da più utenti.
PROPONE	Associazione 1 a N tra utente e modifiche perché un utente può proporre più modifiche, ma la stessa modifica non può essere proposta da più utenti.
GENERA	Associazione 1 a 1 tra modifica e notifiche perché una modifica genera una notifica e la notifica è generata da una sola modifica.
RICEVE	Associazione 1 a N tra utente e notifiche perché un utente può ricevere più di una notifica, ma la stessa notifica non può arrivare a più di un utente.
CONTIENE	Associazione 1 a N tra pagine e versioni perché una pagina può contenere più versioni, ma la versione si riferisce solo ad una pagina.
RACCHIUDE	Associazione 1 a N tra pagine e frasi perché una pagina può contenere più frasi, ma quella stessa frase si riferisce ad una sola pagina.
FORMA	Associazione 1 a N tra frasi e collegamenti perché una frase può generare un collegamento a più pagine ma quello stesso collegamento si può riferire ad una sola pagina.
POSSEGGONO	Associazione 1 a N tra pagine e collegamenti perché una pagina può possedere più collegamenti ma lo stesso collegamento non si definisce per più pagine.

PUBBLICA	Associazione 1 a N tra utente e versioni, perché un utente può creare più versioni, ma una versione non può essere creata da più utenti.
----------	--

2 Ristrutturazione

2.1 Analisi ridondanze

Tutte le volte che una pagina viene creata vengono estrapolate dal testo varie frasi che, se poste tra parentesi quadre [], diventeranno frasi collegate, queste ultime sono definite all'interno dell'entità frasi ma questo potrebbe creare una ridondanza nella tabella collegamenti che ci è comunque molto utile per salvare lo storico di tutti i collegamenti tra pagine.

2.2 Analisi delle generalizzazioni

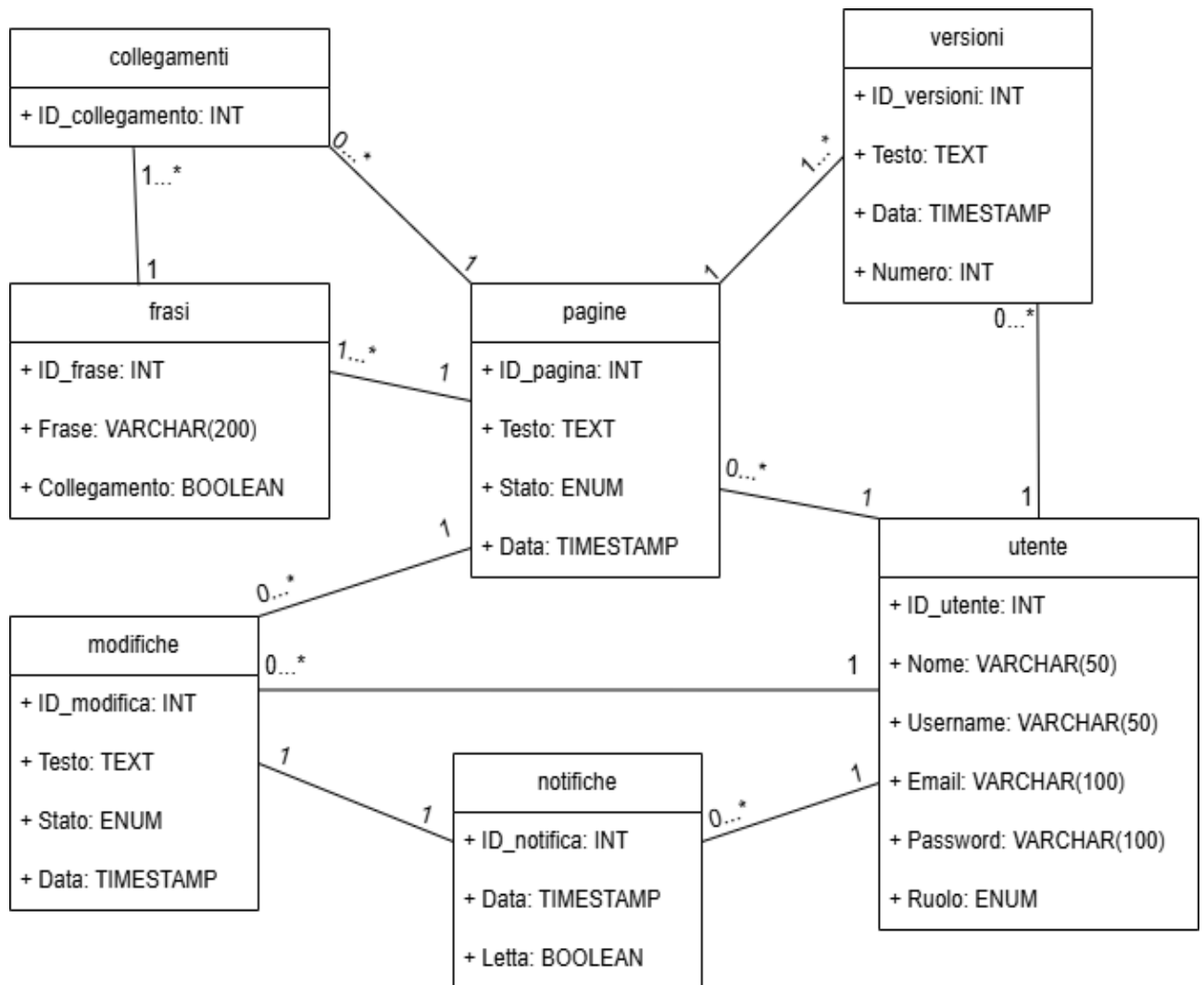
La generalizzazione presente è quella tra **UTENTE** e il suo ruolo ovvero Scrittore o Visitatore, quest'ultima è una relazione totale, disgiunta che nella ristrutturazione del modello ho deciso di inserire come attributo di tipo **ENUM** nella tabella dell'entità **UTENTE**, in modo che quell'attributo avrà la possibilità di ricevere come valore solo Scrittore o Visitatore .

Utente – Scrittore, Visitatore.

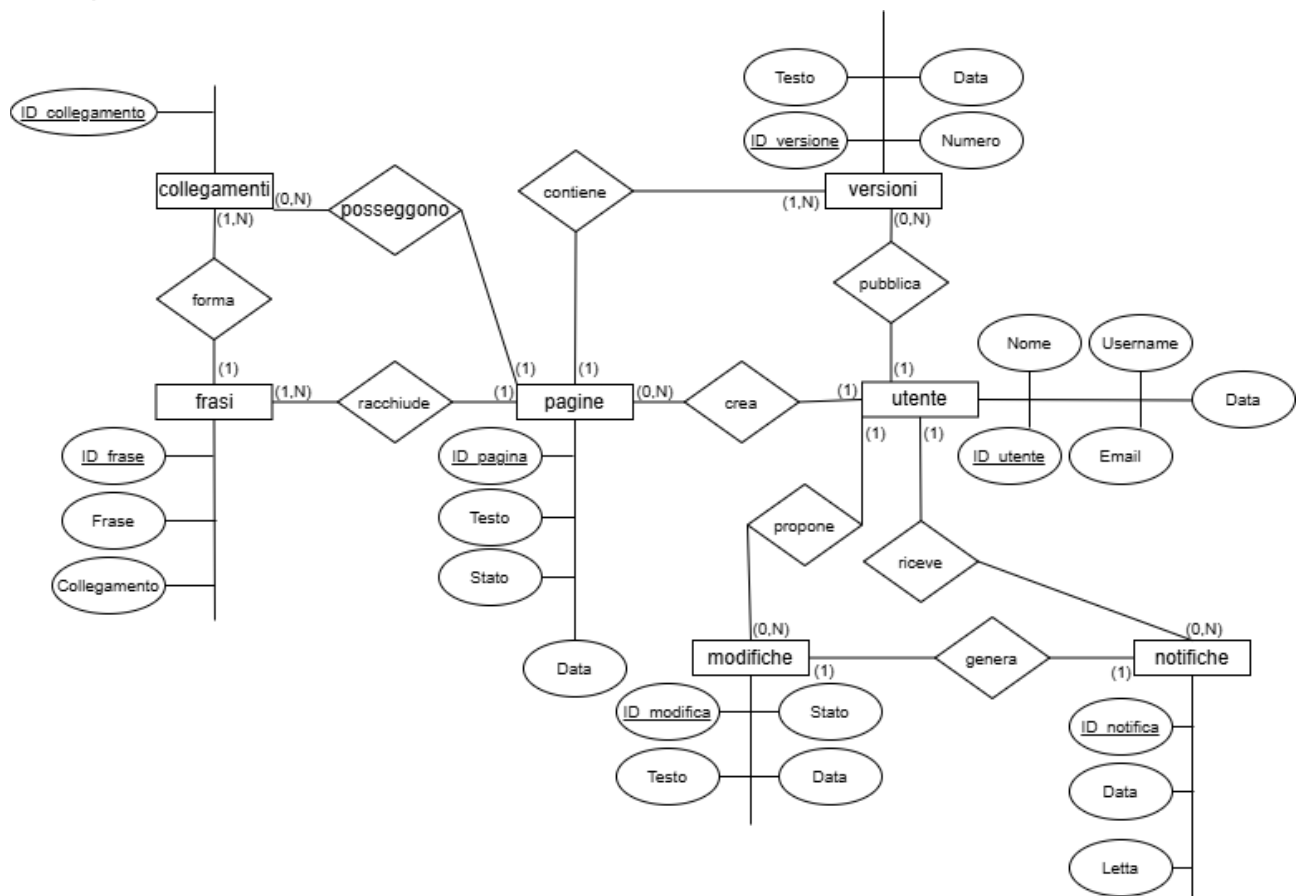
2.3 Chiavi primarie

- Utente - ID_utente;
- Pagine - ID_pagina;
- Modifiche - ID_modifica;
- Versioni - ID_versione;
- Frasi – ID_frase;
- Collegamenti – ID_collegamento;
- Notifiche – ID_notifica.

2.4 Diagramma UML



2.5 Diagramma E-R



2.6 Dizionario classi

CLASSI	DESCRIZIONE	ATTRIBUTI
UTENTE	Contiene tutte le informazioni che riguardano l'utente.	- ID_utente; - Nome; - Username; - Email; - Password; - Ruolo.
PAGINE	Contiene tutte le informazioni che riguardano le pagine.	- ID_pagina; - Titolo; - Testo; - Data.
MODIFICHE	Contiene tutte le informazioni che riguardano le modifiche proposte, accettate e rifiutate.	- ID_modifiche; - Testo;

		• Stato.
VERSIONI	Contiene tutte le informazioni che riguardano la versione attuale della pagina e le vecchie versioni.	• ID_versione; • Testo; • Data; • Numero.
FRASI	Contiene tutte le informazioni che riguardano le frasi ottenute dal testo di una pagina.	• ID_frase; • Frase; • Collegamento.
COLLEGAMENTI	Contiene tutte le informazioni che riguardano i collegamenti tra le pagine tramite le frasi.	• ID_collegamento.
NOTIFICHE	Contiene tutte le informazioni che riguardano le notifiche inviate agli utenti.	• ID_notifica; • Data; • Letta.

2.7 Dizionario associazioni

CLASSI	ASSOCIAZIONE	DESCRIZIONE
UTENTE-PAGINE	CREA	La relazione è 1 a N dove un utente può creare da 0 a molte pagine.
UTENTE-MODIFICHE	PROPONE	La relazione è 1 a N dove un utente può proporre da 0 a molte pagine.
MODIFICHE-NOTIFICHE	GENERA	La relazione è 1 a 1 dove una modifica genera una sola notifica.
UTENTE-NOTIFICHE	RICEVE	La relazione è 1 a N perché un utente può ricevere da 0 a più notifiche.
PAGINE-VERSIONI	CONTIENE	La relazione è 1 a N dove una pagina può contenere 1 o più versioni.
PAGINE-FRASI	RACCHIUDE	La relazione è 1 a N dove una pagina può racchiudere 1 a più frasi.
FRASI-COLLEGAMENTI	FORMA	La relazione è 1 a N dove una frase può formare 1 o più collegamenti.
PAGINE-COLLEGAMENTI	POSSEGGONO	La relazione è 1 a N dove una pagina può possedere 0 o più collegamenti.
UTENTE-VERSIONI	PUBBLICA	La relazione 1 a N dove un utente può creare 0 o più versioni.

3 Modello logico

Per identificare la chiave primaria l'attributo sarà in **grassetto**, invece per identificare le chiavi esterne l'attributo sarà sottolineato.

UTENTE → (**ID_utente**, Nome, Username, Email, Password, Ruolo)

PAGINE → (**ID_pagina**, Titolo, Testo, Data, ID_autore)

ID_autore → ID_utente.utente

MODIFICHE → (**ID_modifica**, Testo, Stato, Data, ID_utente, ID_pagina)

ID_utente → ID_utente.utente

ID_pagina → ID_pagina.pagine

VERSIONI → (**ID_versione**, Testo, Data, Numero, ID_utente, ID_pagina, ID_autore)

ID_utente_modifica → ID_utente.utente

ID_pagina → ID_pagina.pagine

ID_autore → ID_utente.utente

FRASI → (**ID_frase**, Frase, Collegamento, ID_pagina, ID_pagina_collegata)

ID_pagina → ID_pagina.pagine

ID_pagina_collegata → ID_pagina.pagina

COLLEGAMENTO → (**ID_collegamento**, ID_frase_collegamento, ID_pagina_destinazione)

ID_frase_collegamento → ID_frase.frase

ID_pagina_destinazione → ID_pagina.pagina

NOTIFICHE → (**ID_notifica**, Data, Letta, ID_modificatore, ID_autore, ID_creatore_pagina)

ID_modificatore → ID_modifica.modifiche

ID_autore → ID_utente.utente

ID_creatore_pagina → ID_utente.utente

4 Progettazione fisica

4.1 Dizionario dei vincoli

TIPO	ATTRIBUTI	DESCRIZIONE
NULL	Email; Password;	Questi sono gli unici attributi che, come valore, possono assumere NULL tutti gli altri attributi non possono assumere valore NULL.
DEFAULT	Ruolo (Visitatore); Data (now()); Stato (proposta); Collegamento (false); Letta (false);	Questi valori prima di essere modificati, assumeranno come valore di default il valore inserito tra le parentesi tonde accanto agli attributi.
ENUM	Ruolo	Questo attributo può assumere solo due valori ovvero Scrittore e Visitatore.

4.2 Creazione database

Per creare il database, in questo caso il database Wiki1, all'interno del prompt di mysql va inserito il comando:

```
CREATE DATABASE wiki1;
```

Questo comando ci permette la creazione del database che inizialmente sarà vuoto.

4.3 Creazione tabelle

4.3.1 Tabella utente

```
CREATE TABLE utente(  
  ID_utente INT PRIMARY KEY AUTO_INCREMENT,  
  Nome VARCHAR(50) NOT NULL,  
  Username VARCHAR(50) NOT NULL UNIQUE,  
  Email VARCHAR(100),  
  Password VARCHAR(100),  
  Ruolo ENUM('Scrittore', 'Visitatore') DEFAULT 'Visitatore'  
);
```

Questa è la tabella principale del nostro database, questa tabella permette il funzionamento totale di quest'ultimo perché gli utenti creano e modificano le pagine.

Come dominio in questa tabella abbiamo che il ruolo dell'utente può essere solo Scrittore e Visitatore, quest'ultimo viene gestito tramite il tipo dell'attributo ovvero ENUM che può assumere solo il valore di uno di questi due ruoli.

4.3.1.1 Ruoli e permessi

RUOLO	PERMESSI
Visitatore	Visualizzazione pagine
Scrittore	Creazione pagina, modifica pagina, proporre modifica, eliminazione pagina.

4.3.2 Tabella pagine

```
CREATE TABLE pagine(  
  ID_pagina INT PRIMARY KEY AUTO_INCREMENT,  
  Titolo VARCHAR(100) NOT NULL,  
  Testo TEXT NOT NULL,  
  Data TIMESTAMP NOT NULL DEFAULT now(),  
  ID_autore INT NOT NULL,  
  FOREIGN KEY (ID_autore) REFERENCES utente(ID_utente) ON DELETE CASCADE  
);
```

Nella tabella pagine possono essere inseriti dei valori solo dagli utenti con il ruolo Scrittore, la data di creazione di ogni pagina è settata come tipo di attributo TIMESTAMP con clausola NOW() in modo che nel momento in cui viene creata una pagina la data sarà inserita automaticamente e non dovrà essere inserita manualmente.

4.3.3 Tabella modifiche

```
CREATE TABLE modifiche(
```

```

ID_modifica INT PRIMARY KEY AUTO_INCREMENT,
Testo TEXT,
Stato ENUM('Proposta', 'Approvata', 'Rifiutata') NOT NULL DEFAULT 'Proposta',
ID_utente INT NOT NULL,
ID_pagina INT NOT NULL,
Data TIMESTAMP NOT NULL DEFAULT now(),
FOREIGN KEY (ID_utente) REFERENCES utente(ID_utente) ON DELETE CASCADE,
FOREIGN KEY (ID_pagina) REFERENCES pagine(ID_pagina) ON DELETE CASCADE
);

```

Le modifiche possono essere proposte solo da utenti scrittori e possono essere proposte su qualunque pagina ma, se la modifica viene proposta dal creatore della pagina la pagina verrà modificata automaticamente.

4.3.4 Tabella versioni

```

CREATE TABLE versioni(
    ID_versione INT PRIMARY KEY AUTO_INCREMENT,
    Testo TEXT NOT NULL,
    Data TIMESTAMP NOT NULL DEFAULT now(),
    Numero INT NOT NULL,
    ID_pagina INT NOT NULL,
    ID_autore INT NOT NULL,
    ID_utente_modifica INT NOT NULL,
    FOREIGN KEY (ID_pagina) REFERENCES pagine(ID_pagina) ON DELETE CASCADE,
    FOREIGN KEY (ID_autore) REFERENCES utente(ID_utente) ON DELETE CASCADE
    FOREIGN KEY (ID_utente_modifica) REFERENCES utente(ID_utente) ON DELETE CASCADE
);

```

La tabella versioni viene popolata automaticamente ogni volta che una pagina viene creata o modificata.

4.3.5 Tabella frasi

```
CREATE TABLE frasi(  
    ID_frase INT PRIMARY KEY AUTO_INCREMENT,  
    Frase VARCHAR(200) NOT NULL,  
    Collegamento BOOLEAN DEFAULT FALSE,  
    ID_pagina INT NOT NULL,  
    ID_pagina_collegata INT,  
    FOREIGN KEY (ID_pagina) REFERENCES pagine(ID_pagina) ON DELETE CASCADE  
    FOREIGN KEY (ID_pagina_collegata) REFERENCES pagine(ID_pagina) ON DELETE CASCADE  
);
```

La tabella frasi viene popolata automaticamente ogni volta che una pagina viene creata le frasi vengono estrapolate come frasi normali quelle tra le punteggiature e le frasi tra le parentesi quadre come frasi collegate.

4.3.6 Tabella collegamenti

```
CREATE TABLE collegamenti(  
    ID_collegamento INT PRIMARY KEY AUTO_INCREMENT,  
    ID_frase_collegamento INT NOT NULL,  
    ID_pagina_destinazione INT NOT NULL,  
    FOREIGN KEY (ID_frase_collegamento) REFERENCES frasi(ID_frase) ON DELETE CASCADE,  
    FOREIGN KEY (ID_pagina_destinazione) REFERENCES pagine(ID_pagina) ON DELETE CASCADE  
);
```

La tabella collegamenti viene popolata automaticamente quando in una pagina sono presenti nel testo delle frasi con le parentesi quadre e collega la frase alla pagina con il titolo uguale alla frase.

4.3.7 Tabella notifiche

```
CREATE TABLE notifiche(  
    ID_notifica INT AUTO_INCREMENT PRIMARY KEY,  
    Data TIMESTAMP NOT NULL DEFAULT now(),  
    Letta BOOLEAN NOT NULL DEFAULT FALSE,  
    ID_modificatore INT NOT NULL UNIQUE,  
    ID_autore INT NOT NULL,  
    ID_creatore_pagina INT NOT NULL,  
    FOREIGN KEY (ID_modificatore) REFERENCES modifiche(ID_modifica) ON DELETE CASCADE,  
    FOREIGN KEY (ID_autore) REFERENCES utente(ID_utente) ON DELETE CASCADE  
    FOREIGN KEY (ID_creatore_pagina) REFERENCES utente(ID_utente) ON DELETE CASCADE  
);
```

La tabella notifiche viene popolata ogni volta che una modifica viene proposta ed invia una notifica all'utente creatore della pagina.

4.4 Trigger ed event

Nel database ci sono vari trigger ed eventi che permettono il corretto funzionamento di tutto il database, evitando che utenti senza permessi abbiano la possibilità di creare, eliminare e modificare pagine. I trigger gestiscono anche il popolamento automatico della pagina.

4.4.1 Trigger per proposta modifica

```
DELIMITER //
```

```
CREATE TRIGGER propostaModifica
```

AFTER INSERT ON modifiche

FOR EACH ROW

BEGIN

DECLARE v_autore INT DEFAULT 0;

SELECT ID_autore INTO v_autore FROM pagine WHERE ID_pagina = NEW.ID_pagina;

INSERT INTO notifiche (Data, ID_modificatore, ID_autore, ID_creatore_pagina)

VALUES (now(), NEW.ID_modifica, ID_utente, v_autore);

END //

DELIMITER;

Questo trigger si attiva ogni volta che viene proposta una nuova modifica e si occupa di inviare una notifica con l'ID del creatore della modifica proposta.

4.4.2 Trigger accettazione e rifiuto modifica

DELIMITER //

CREATE TRIGGER accetta_rifiuta_modifica

AFTER UPDATE ON modifiche

FOR EACH ROW

BEGIN

DECLARE numero_versioni INT;

DECLARE v_autore INT;

SELECT COUNT(*) INTO numero_versioni FROM versioni WHERE ID_pagina = NEW.ID_pagina;

SELECT ID_autore INTO v_autore FROM pagine WHERE ID_pagina = NEW.ID_pagina;

IF NEW.Stato = 'Approvata' THEN

UPDATE pagine SET Testo = NEW.Testo WHERE ID_pagina = NEW.ID_pagina;

```

INSERT INTO versioni (Testo, Data, Numero, ID_pagina, ID_autore, ID_utente_modifica)
VALUES (OLD.Testo, NOW(), numero_versioni + 1, NEW.ID_pagina, v_autore, NEW.ID_utente);

UPDATE notifiche

SET Letta = TRUE

WHERE ID_modificatore = NEW.ID_modifica;

END IF;

```

```

IF NEW.Stato = 'Rifiutata' THEN

    UPDATE notifiche

    SET Letta = TRUE

    WHERE ID_modificatore = NEW.ID_modifica;

END IF;

END //

```

```

DELIMITER ;

```

Questo trigger si attiva dopo che viene aggiornata la tabella modifiche, si occupa di controllare se la modifica è stata approvata o rifiutata, se la modifica viene approvata la pagina viene sostituita con la nuova versione, la vecchia versione viene inserita nella tabella versioni e nella tabella notifiche viene impostato l'attributo **Letta** su true, invece se la modifica viene rifiutata resta tutto invariato e anche in questo caso l'attributo Letta viene impostato su true.

4.4.3 Trigger controllo dati utente

```

DELIMITER //

```

```

CREATE TRIGGER check_password_email

BEFORE INSERT

ON utente

FOR EACH ROW

```

```

BEGIN

IF NEW.Ruolo = 'Scrittore' THEN

    IF NEW.Email IS NULL OR NEW.Email = " THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sei uno scrittore inserisci la email';

    END IF;

    IF NEW.Password IS NULL OR NEW.Password = " THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Sei uno scrittore inserisci la password';

    END IF;

END IF;

END//

DELIMITER ;

```

Questo trigger si attiva prima dell'inserimento dei valori nella tabella utente e controlla se il ruolo viene definito come scrittore allora devono essere inseriti i valori anche negli attributi email e password in caso contrario da un messaggio di errore in entrambi i casi.

4.4.4 Trigger controllo ruolo per modifica

```

DELIMITER //

CREATE TRIGGER check_ruolo_modifica

BEFORE INSERT

ON modifiche

FOR EACH ROW

BEGIN

    DECLARE ruolo_utente ENUM('Scrittore', 'Visitatore');

    SELECT Ruolo INTO ruolo_utente FROM utente WHERE NEW.ID_utente = ID_utente;

    IF ruolo_utente = 'Visitatore' THEN

        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Non hai i permessi per apportare o proporre delle modifiche';

    END IF;

END//

```

END//

DELIMITER;

Questo trigger si attiva prima dell'inserimento dei valori nella tabella modifica e controlla se l'utente che propone la modifica è scrittore, se non lo è allora non permette l'inserimento dei valori.

4.4.5 Trigger collegamento e controllo frasi inserimento

DELIMITER //

CREATE TRIGGER estrai_frase_a

AFTER INSERT ON pagine

FOR EACH ROW

BEGIN

DECLARE pos INT DEFAULT 1;

DECLARE fine INT;

DECLARE frase VARCHAR(1000);

DECLARE start_collegamento INT;

DECLARE end_collegamento INT;

WHILE pos <= CHAR_LENGTH(NEW.Testo) DO

SET fine = LEAST(

COALESCE(NULLIF(LOCATE('.', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),

COALESCE(NULLIF(LOCATE('!', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),

COALESCE(NULLIF(LOCATE('?', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),

COALESCE(NULLIF(LOCATE(':', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),

COALESCE(NULLIF(LOCATE(',', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),

COALESCE(NULLIF(LOCATE('[', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1)

);

SET frase = TRIM(SUBSTRING(NEW.Testo, pos, fine - pos));

```

IF LENGTH(frase) > 0 THEN
    INSERT INTO frasi (Frase, Collegamento, ID_pagina, ID_pagina_collegata)
    VALUES (frase, FALSE, NEW.ID_pagina, NULL);
END IF;

IF MID(NEW.Testo, fine, 1) = '[' THEN
    SET start_collegamento = fine;
    SET end_collegamento = LOCATE(']', NEW.Testo, start_collegamento);

    IF end_collegamento > start_collegamento THEN
        SET frase = SUBSTRING(NEW.Testo, start_collegamento + 1, end_collegamento -
start_collegamento - 1);

        IF LENGTH(frase) > 0 THEN
            INSERT INTO frasi (Frase, Collegamento, ID_pagina, ID_pagina_collegata)
            VALUES (frase, TRUE, NEW.ID_pagina, NULL);
        END IF;

        SET pos = end_collegamento + 1;
    END IF;
ELSE
    SET pos = fine + 1;
END IF;
END WHILE;
END //

DELIMITER ;

```

Questo trigger si attiva dopo l’inserimento dei valori nella tabella pagine e si occupa di popolare la tabella frasi dividendole in collegate

quelle tra le parentesi quadre e in normali quelle tra tutti gli altri segni di punteggiatura.

4.4.6 Trigger collegamento e controllo frasi aggiornamento

```
DELIMITER //
```

```
CREATE TRIGGER estrai_frase_ag
```

```
AFTER UPDATE ON pagine
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE pos INT DEFAULT 1;
```

```
    DECLARE fine INT;
```

```
    DECLARE frase VARCHAR(1000);
```

```
    DECLARE start_collegamento INT;
```

```
    DECLARE end_collegamento INT;
```

```
    WHILE pos <= CHAR_LENGTH(NEW.Testo) DO
```

```
        SET fine = LEAST(
```

```
            COALESCE(NULLIF(LOCATE('.', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),
```

```
            COALESCE(NULLIF(LOCATE('!', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),
```

```
            COALESCE(NULLIF(LOCATE('?', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),
```

```
            COALESCE(NULLIF(LOCATE(';', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),
```

```
            COALESCE(NULLIF(LOCATE(',', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1),
```

```
            COALESCE(NULLIF(LOCATE('[', NEW.Testo, pos), 0), CHAR_LENGTH(NEW.Testo) + 1)
```

```
        );
```

```
        SET frase = TRIM(SUBSTRING(NEW.Testo, pos, fine - pos));
```

```
        IF LENGTH(frase) > 0 THEN
```

```
            INSERT INTO frasi (Frase, Collegamento, ID_pagina, ID_pagina_collegata)
```

```
            VALUES (frase, FALSE, NEW.ID_pagina, NULL);
```

```
        END IF;
```

```

IF MID(NEW.Testo, fine, 1) = '[' THEN

    SET start_collegamento = fine;

    SET end_collegamento = LOCATE(']', NEW.Testo, start_collegamento);

    IF end_collegamento > start_collegamento THEN

        SET frase = SUBSTRING(NEW.Testo, start_collegamento + 1, end_collegamento -
start_collegamento - 1);

        IF LENGTH(frase) > 0 THEN

            INSERT INTO frasi (Frase, Collegamento, ID_pagina, ID_pagina_collegata)

            VALUES (frase, TRUE, NEW.ID_pagina, NULL);

        END IF;

        SET pos = end_collegamento + 1;

    END IF;

ELSE

    SET pos = fine + 1;

END IF;

END WHILE;

END //

DELIMITER ;

```

Questo trigger si attiva dopo la modifica dei valori nella tabella pagine e si occupa di popolare la tabella frasi dividendole in collegate quelle tra le parentesi quadre e in normali quelle tra tutti gli altri segni di punteggiatura.

4.4.7 Trigger per controllo cronologia modifiche

```
CREATE TRIGGER controllo_modifica_precedente
BEFORE UPDATE ON modifiche
FOR EACH ROW
BEGIN
    DECLARE num_modifiche INT;
    DECLARE id_modifica_min INT;
    DECLARE messaggio_errore VARCHAR(255);
    SELECT COUNT(*) INTO num_modifiche
    FROM modifiche
    WHERE ID_pagina = NEW.ID_pagina AND Stato = 'Proposta' AND Data < NEW.Data;

    SELECT MIN(ID_modifica) INTO id_modifica_min
    FROM modifiche
    WHERE ID_pagina = NEW.ID_pagina AND Stato = 'Proposta' AND Data < NEW.Data;

    SET messaggio_errore = CONCAT('Ci sono modifiche proposte precedentemente a questa,
dovresti iniziare da quelle. ID della più vecchia è: ', id_modifica_min);

    IF num_modifiche > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = messaggio_errore;
    END IF;
END //

DELIMITER ;
```

Questo trigger si attiva prima di una di una modifica dei valori nella tabella modifiche e fa in modo che tutte le modifiche siano revisionate e quindi accettate o rifiutate in ordine cronologico dalla più vecchia alla più recente.

4.4.8 Trigger per controllo proprietario pagina

```
DELIMITER //

CREATE TRIGGER controlla_proprietario_pagina
BEFORE INSERT ON modifiche
FOR EACH ROW
BEGIN
    DECLARE proprietario BOOLEAN;

    SELECT COUNT(*) INTO proprietario
    FROM pagine
    WHERE ID_pagina = NEW.ID_pagina AND ID_autore = NEW.ID_utente;

    IF proprietario > 0 THEN
        SET NEW.Stato = 'Approvata';
    END IF;
END //

DELIMITER ;
```

Questo trigger si attiva prima dell'inserimento dei valori nella tabella notifiche e fa in modo che se chi inserisce la proposta di modifica è il proprietario della pagina, allora la modifica non deve essere approvata e la pagina si aggiorna immediatamente.

4.4.9 Trigger per gestione modifica da proprietario

```
DELIMITER //

CREATE TRIGGER modifica_proprietario
AFTER INSERT ON modifiche
```

```

FOR EACH ROW
BEGIN
    DECLARE numero_versioni INT;
    DECLARE v_autore INT;
    SELECT COUNT(*) INTO numero_versioni FROM versioni WHERE ID_pagina = NEW.ID_pagina;

    SELECT ID_autore INTO v_autore FROM pagine WHERE ID_pagina = NEW.ID_pagina;

    IF NEW.Stato = 'Approvata' THEN
        UPDATE pagine SET Testo = NEW.Testo WHERE ID_pagina = NEW.ID_pagina;

        INSERT INTO versioni (Testo, Data, Numero, ID_pagina, ID_autore, ID_utente_modifica)
        VALUES (NEW.Testo, NOW(), numero_versioni + 1, NEW.ID_pagina, v_autore,
        NEW.ID_utente);

        UPDATE notifiche
        SET Letta = TRUE
        WHERE ID_modificatore = NEW.ID_modifica;
    END IF;
END //
DELIMITER ;

```

Questo trigger si attiva dopo l’inserimento dei valori su modifiche e fa in modo che, se la modifica viene proposta dal proprietario allora l’attributo Letta di notifiche viene impostato automaticamente su true e la vecchia pagina non viene sovrascritta ma viene archiviata in versioni.

4.4.10 Trigger per creazione versione pagina N1

```

DELIMITER //

```

```

CREATE TRIGGER versione1
AFTER INSERT ON pagine
FOR EACH ROW
BEGIN
    DECLARE v_autore INT;

    SET v_autore = NEW.ID_autore;

    INSERT INTO versioni (Testo, Data, Numero, ID_pagina, ID_autore)
    VALUES (NEW.Testo, NOW(), 1, NEW.ID_pagina, v_autore);
END //

DELIMITER ;

```

Questo trigger si attiva dopo l’inserimento dei valori in pagine e si occupa di creare la prima versione della pagina nella tabella di archivio versioni.

4.4.11 Event creazione collegamenti

```

DELIMITER //

CREATE EVENT aggiorna_collegamenti_frase
ON SCHEDULE EVERY 1 MINUTE
DO
BEGIN
    UPDATE frase f
    JOIN pagine p ON LOWER(TRIM(f.Frase)) = LOWER(TRIM(p.Titolo))
    SET f.ID_pagina_collegata = p.ID_pagina
    WHERE f.Collegamento = 1 AND f.ID_pagina_collegata IS NULL;

```

END //

DELIMITER ;

Questo evento si attiva ogni minuto e fa in modo di collegare tutte le pagine dove la frase collegata è uguale al titolo della pagina di destinazione, non facendo caso alle maiuscole e le minuscole.

4.4.12 Event aggiornamento storico collegamenti

DELIMITER //

```
CREATE EVENT aggiungi_collegamenti  
ON SCHEDULE EVERY 1 MINUTE  
DO  
BEGIN  
  INSERT INTO collegamenti (ID_frase_collegamento, ID_pagina_destinazione)  
  SELECT f.ID_frase, p.ID_pagina  
  FROM frasi f  
  JOIN pagine p ON f.ID_pagina_collegata = p.ID_pagina  
  WHERE f.ID_pagina_collegata IS NOT NULL  
  AND NOT EXISTS (  
    SELECT 1 FROM collegamenti c  
    WHERE c.ID_frase_collegamento = f.ID_frase  
    AND c.ID_pagina_destinazione = p.ID_pagina  
  );  
END //
```

DELIMITER ;

Questo evento si attiva ogni minuto e a differenza dell'event precedente, quest'ultimo si occupa di aggiornare lo storico dei collegamenti nella tabella archivio collegamenti.

4.4.13 Event controllo pagina cancellata

DELIMITER //

```
CREATE EVENT controlla_pagine_collegate  
ON SCHEDULE EVERY 1 MINUTE  
DO  
BEGIN  
    UPDATE frasi f  
    LEFT JOIN pagine p ON f.ID_pagina_collegata = p.ID_pagina  
    SET f.ID_pagina_collegata = NULL  
    WHERE p.ID_pagina IS NULL;  
END //
```

DELIMITER ;

Questo trigger si attiva ogni minuto e controlla se una pagina collegata viene eliminata in tal caso imposta pagina collegata su NULL.

4.4.14 Event controllo ruolo utente sbagliato

DELIMITER //

CREATE EVENT controlla_utente_scrittore

ON SCHEDULE EVERY 1 minute

DO

BEGIN

UPDATE utente

SET Ruolo = 'Scrittore'

WHERE Password IS NOT NULL AND Email IS NOT NULL;

END //

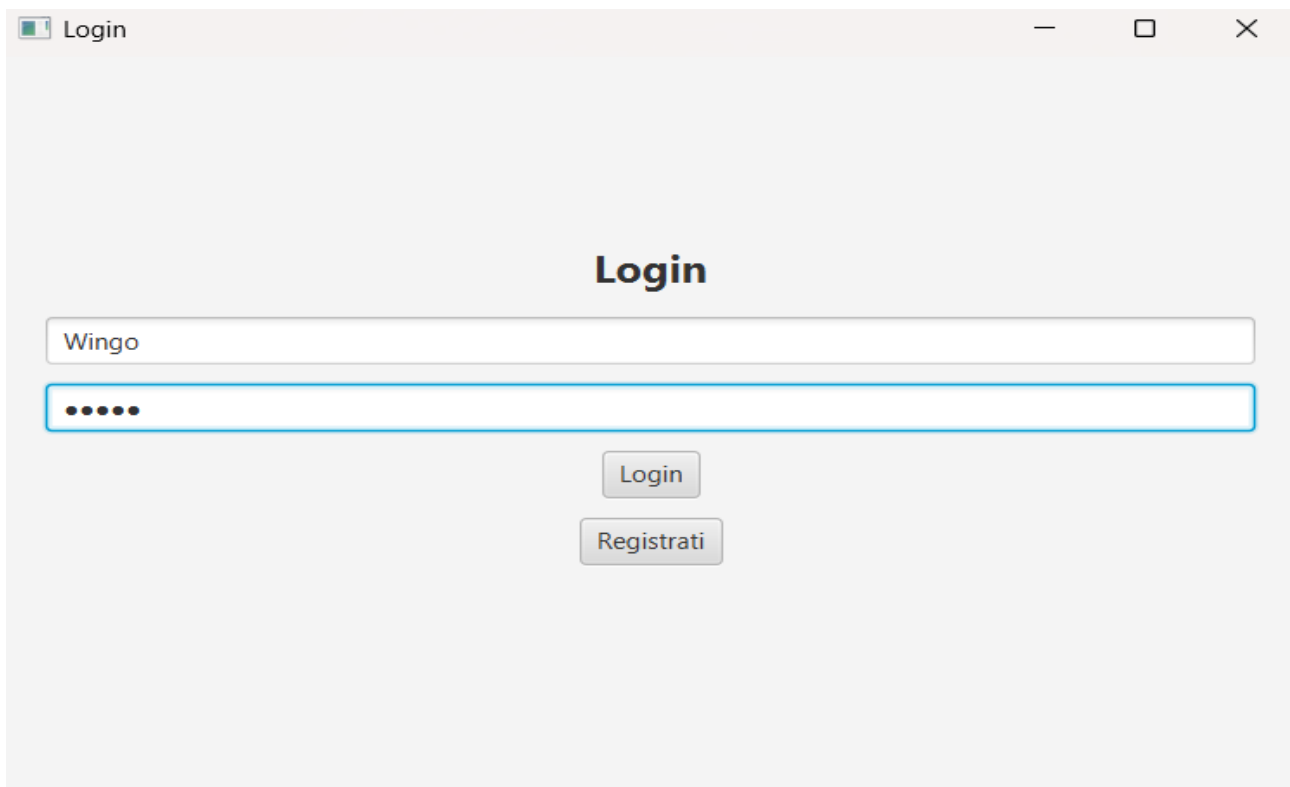
DELIMITER ;

Questo event si attiva ogni minuto e controlla se un utente ha inserito sia password sia email ma ha sbagliato ad inserire il proprio ruolo impostandolo come Visitatore e non come Scrittore, allora il ruolo viene automaticamente impostato come Scrittore.

5 APPLICATIVO

5.1 Login

Nella schermata di login abbiamo due possibilità ovvero, immettere nel caso dei Visitatori solo l'username e loggarci, immettere nel caso degli Scrittori username e password e loggarci, oppure registrarci.

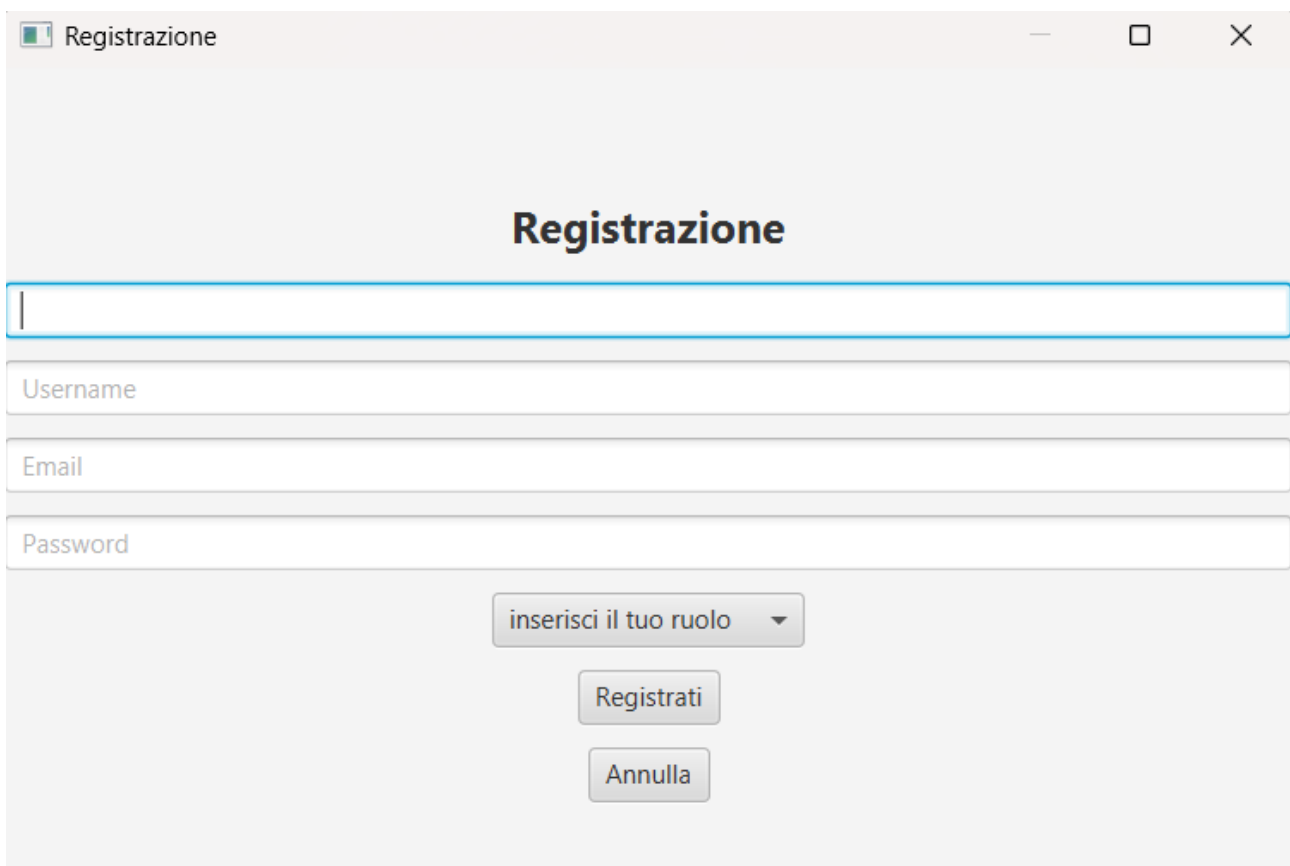


The screenshot shows a web application window titled "Login". The window has a light gray background and a title bar with standard window controls (minimize, maximize, close). The main content area is centered and contains the following elements:

- A heading "Login" in bold black text.
- A text input field containing the username "Wingo".
- A password input field with five dots representing masked characters.
- A "Login" button located below the password field.
- A "Registrati" button located below the "Login" button.

5.2 Registrazione

Se clicchiamo su registrati ci aprirà questa schermata che ci permette di inserire tutti i dati, rispettando ovviamente tutti i vincoli dati dal database, nel caso in cui l'utente dovesse immettere come ruolo Visitatore anche se ha inserito Email e password il database si occuperà di inserirlo in automatico nel ruolo dello Scrittore, una volta completata la registrazione l'utente sarà salvato nel database e potrà loggarsi



Registrazione

Registrazione

Username

Email

Password

inserisci il tuo ruolo ▼

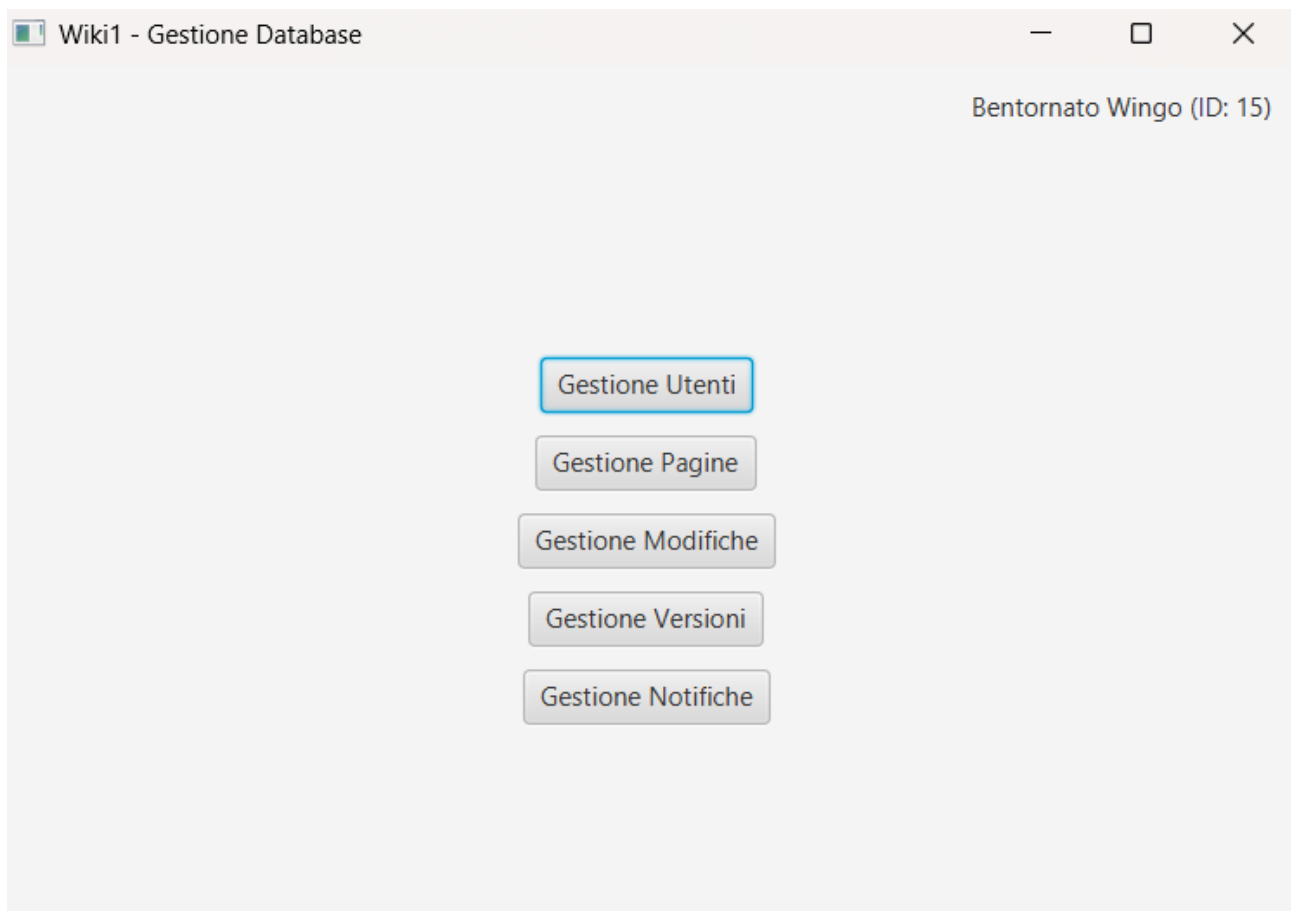
Registrati

Annulla

5.3 Menu

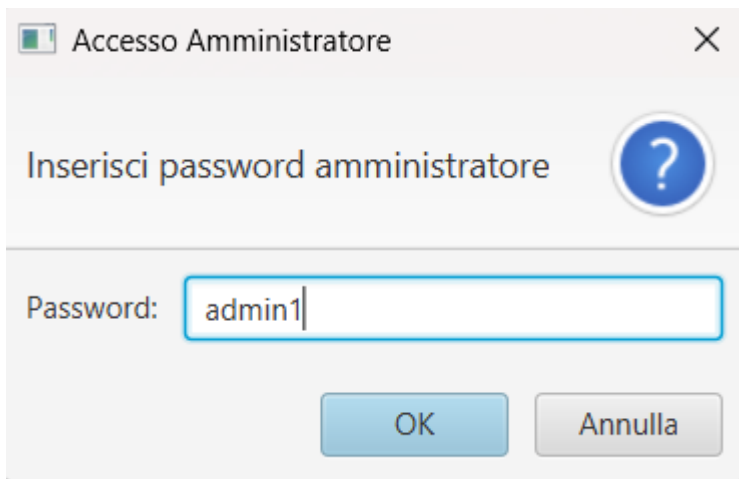
Una volta loggati, ci porterà nella pagina del menu dove troveremo tutte le nostre componenti della Wiki.

Oltre a tutti i componenti della wiki troveremo anche Il nostro Username e il nostro ID in alto a destra.



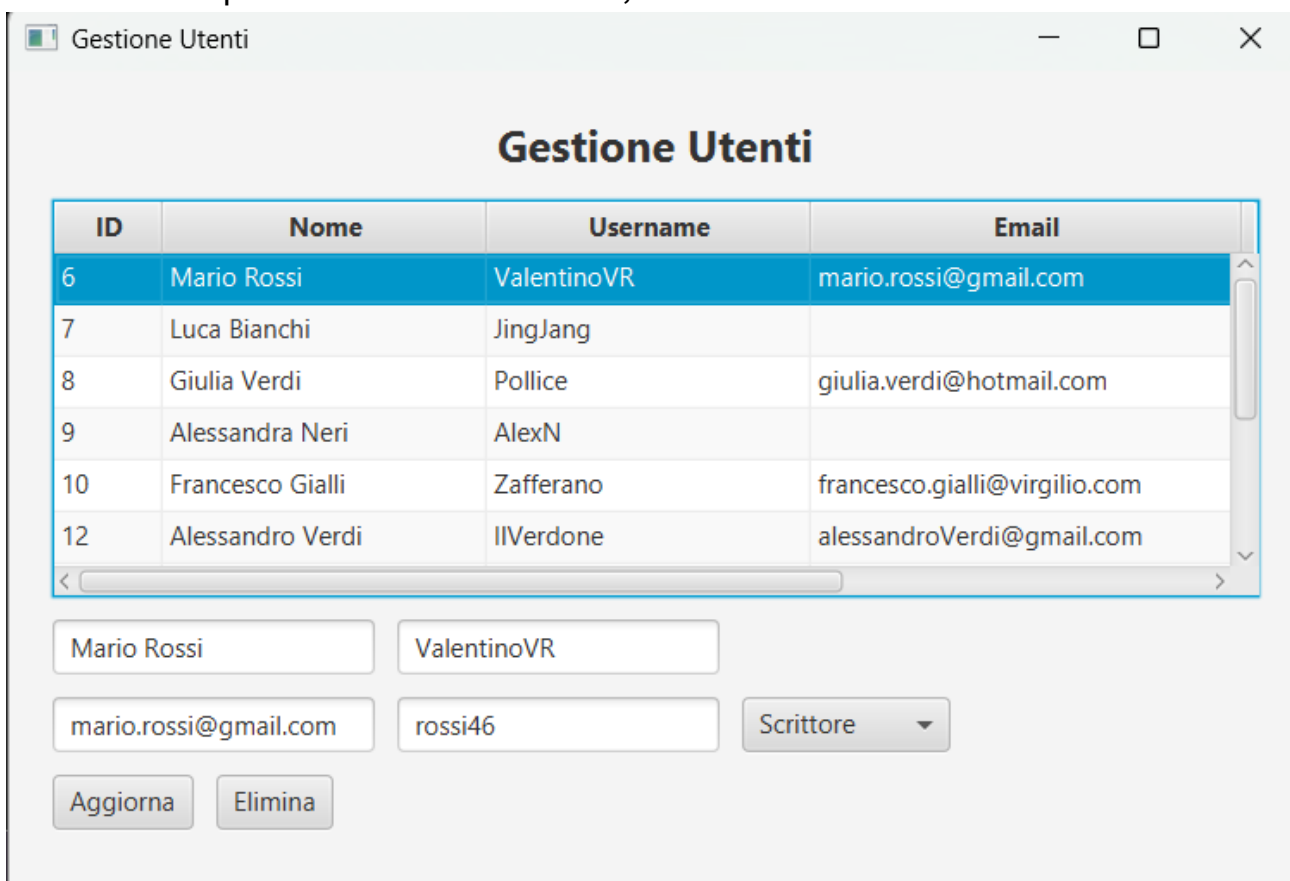
5.4 Utente

Se clicchiamo su utente ci chiederà di inserire una password admin in modo che solo chi è autorizzato può controllare quanti utenti sono registrati alla nostra wiki.



A dialog box titled "Accesso Amministratore" with a close button (X) in the top right corner. Below the title bar, the text "Inserisci password amministratore" is displayed next to a blue circular help icon containing a question mark. A text input field labeled "Password:" contains the text "admin1". At the bottom, there are two buttons: "OK" and "Annulla".

Dopo aver inserito la password correttamente allora accederemo alla sezione utente che ci permetterà di modificare, eliminare o visualizzare un utente.



A web interface titled "Gestione Utenti" with standard window controls (minimize, maximize, close) in the top right. Below the title, there is a table with four columns: ID, Nome, Username, and Email. The table contains six rows of user data. Below the table, there are input fields for editing the selected user (Mario Rossi), a "Scrittore" dropdown menu, and "Aggiorna" and "Elimina" buttons.

ID	Nome	Username	Email
6	Mario Rossi	ValentinoVR	mario.rossi@gmail.com
7	Luca Bianchi	JingJang	
8	Giulia Verdi	Pollice	giulia.verdi@hotmail.com
9	Alessandra Neri	AlexN	
10	Francesco Gialli	Zafferano	francesco.gialli@virgilio.com
12	Alessandro Verdi	IlVerdone	alessandroVerdi@gmail.com

Mario Rossi ValentinoVR

mario.rossi@gmail.com rossi46 Scrittore ▼

Aggiorna Elimina

5.5 Pagine

Se clicchiamo su pagine ci porterà sulla schermata di visualizzazione di tutte le pagine presenti sulla wiki dove in veste di scrittori possiamo creare una nuova pagina e se siamo i proprietari di una pagina già esistente possiamo eliminarla o modificarla direttamente senza proporre una modifica.

Gestione Pagine

Gestione Pagine

ID Pagina	Titolo	Testo	Autore	Pagina Collegata	
52	Prova	i [dati] all'interno della [tabella] pa...	Wingo	Dati, Tabella	
51	Ciao	Hello my darlin	Wingo		
50	ciao	HELLO THERE, GENERAL KENOBI!!!!...	Pollice		
48	MCU	Marvel dajdioahsifaufg	Zafferano		
40	Introduzione al Database	Ciao mi chiamo Marco	ValentinoVR		
41	Dati	Le informazioni memorizzate nel d...	ValentinoVR	Record	
42	Record	è la riga di una [tabella] del databa...	Pollice	Tabella	
43	Tabella	HELLO THERE	Zafferano		
44	I droni di Misterio	Ciao mi chiamo marco	Pollice		
45	Misterio	Super Cattivo della [MCU]	Pollice	MCU	

Prova

i [dati] all'interno della [tabella] pagine sono molto grandi.

Aggiungi Pagina

Modifica Pagina

Elimina Pagina

5.6 Notifiche

Se clicchiamo su notifiche ci mostrerà una sezione con tutte le modifiche che ci hanno proposto in ordine cronologico

Gestione Notifiche

Gestione Notifiche

ID Notifica	Data	Letta	ID Modificatore	ID Autore	ID Creatore Pagina	
32	2025-03-12 11:30:53.0	true	42	15	8	
31	2025-03-12 11:30:30.0	true	41	15	8	
28	2025-03-10 18:32:10.0	true	38	15	8	

5.7 Modifiche

Se clicchiamo su modifiche ci mostra una sezione con tutte le modifiche che ci hanno proposto e che abbiamo proposto, in questa sezione possiamo approvare o rifiutare tutte le modifiche tranne quelle effettuate da noi ovviamente.

Gestione Modifiche

Gestione Modifiche

ID Modifica	Testo	Stato	ID Utente	ID Pagina	Data	Proposta	
42	falso non è una citazione...	Rifiutata	15	50	2025-03-12 11:30:5...	a me	
41	HELLO THERE, GENERAL ...	Approvata	15	50	2025-03-12 11:30:3...	a me	
38	HELLO THERE, GENERAL ...	Approvata	15	50	2025-03-10 18:32:1...	a me	

Approva

Rifiuta

Proponi Nuova Modifica:

Testo della modifica

ID della pagina

Proponi

5.8 Versioni

Se clicchiamo su versioni ci mostra lo storico di tutte le versioni create di tutte le pagine della wiki, se clicchiamo due volte su di una pagina in particolare ci mostra un dettaglio di quest'ultima.

Gestione Versioni

Gestione Versioni

ID Versione	Testo	Numero	Data	ID Pagina	ID Autore
48	HELLO THERE, GENERAL KENO...	5	2025-03-12 11:31:5...	50	8
47	HELLO THERE	10	2025-03-12 11:27:4...	43	10
46	HELLO THERE	10	2025-03-12 11:27:4...	43	10
45	HELLO THERE	9	2025-03-12 11:27:3...	43	10
44	HELLO THERE	8	2025-03-12 11:27:3...	43	10
43	Marvel dajdioahsifaufg	2	2025-03-12 11:27:2...	48	10
39	HELLO THERE, GENERAL KENO...	4	2025-03-10 20:26:3...	50	8
38	HELLO THERE, GENERAL KENO...	3	2025-03-10 20:25:2...	50	8
37	i [dati] all'interno della [tabella...	1	2025-03-10 20:00:2...	52	15
36	HELLO THERE, GENERAL KENO...	2	2025-03-10 18:33:5...	50	8
35	Ciao come stai? mi chiamo [re...	1	2025-03-10 18:00:1...	51	15

Dettaglio Versione - ID: 48

HELLO THERE, GENERAL KENOBI!!! (citazione a star wars)